Shifting Rationales Behind Stable Classifiers: Evaluating Explanation Consistency Under Application-Domain Transfer in SQL Injection Detection

Anonymous Authors

Submitted for Blind Peer Review

Artifact DOI: `10.5281/zenodo.19627322` (Superviz26-SQL Benchmark)

Code Repository: `https://anonymous.4open.science/r/SQLi-Explainability-Transfer`

Abstract

Machine learning models for Structured Query Language injection (SQLi) detection may maintain high predictive performance when transferred between applications while producing different feature attributions. This study investigates this phenomenon using the authentic Superviz26-SQL benchmark, comparing Logistic Regression, Random Forest, and XGBoost models trained on Domain B (Sakila movie rental) with within-domain reference models trained on Domain D (Oracle Human Resources). All models are evaluated on the same 2,000-query Domain D sample, comprising 1,000 benign and 1,000 attack queries (attack prevalence of 50.0%). Because the benchmark's official training partition contains no attack examples (100,000 benign and 0 attack queries per scenario), supervised training and evaluation samples were drawn from the official test partition stream under query-level isolation, with exact-query and normalized-query overlap verified as zero. This sampling protocol and its methodological limitations are explicitly considered in interpreting the results.

Cross-domain F_1-scores were 0.9858 for Logistic Regression, 0.9899 for Random Forest, and 0.9879 for XGBoost, compared to within-domain F_1-scores of 0.9894, 0.9752, and 0.9909, respectively. Defining signed transfer performance difference as \Delta F_1 = F_1(\text{cross-domain } B \to D) - F_1(\text{within-domain } D), Logistic Regression and XGBoost exhibit slight cross-domain performance drops (\Delta F_1 = -0.0036 and -0.0030, respectively), whereas Random Forest exhibits higher cross-domain performance (\Delta F_1 = +0.0147) associated with fewer false alarms on Domain D evaluation records (2 vs. 34 false positives). Comparison of signed SHAP attribution vectors across a stratified 200-query evaluation subset yielded mean cosine similarities of 0.8474, 0.8706, and 0.9058, indicating measurable attribution differences despite high positive directional alignment. Mean Spearman rank correlations between signed attribution values varied substantially across architectures, ranging from 0.4467 (Logistic Regression) and 0.4523 (XGBoost) to 0.8010 (Random Forest), demonstrating that vector direction and feature ranking capture distinct dimensions of explanation agreement. Across this 200-query explanation subset, 97.5%–99.5% of queries exhibited concordant-correct classifications between

cross-domain and within-domain models despite shifted attributions. A template census revealed that all 29 vulnerable Domain D application templates were represented in the 2,000-query evaluation set, rendering a template-disjoint supervised within-domain reference infeasible under the fixed protocol.

The findings provide descriptive evidence that predictive performance alone does not characterize attribution consistency in this transfer setting. Broader conclusions require additional transfer pairs and evaluation protocols that better separate application-domain shift from template exposure and sampling effects.

Keywords: SQL Injection Detection, Explainable AI (XAI), Domain Transfer, SHAP, LIME, Explanation Consistency, Machine Learning Security.

1. Introduction

Structured Query Language injection (SQLi) remains among the most critical threats to web application security, consistently featured in the OWASP Top Ten vulnerabilities [1]. While traditional perimeter defenses rely on signature-based Web Application Firewalls (WAFs), attackers routinely bypass static pattern matchers using evasion techniques such as whitespace obfuscation, inline comment injection, tautological predicates, and character encoding schemes [2]. In response, machine learning (ML) classifiers trained on manual lexical features [3] or token embeddings [4] have emerged as adaptive alternatives for web security inspection.

Despite benchmark accuracies exceeding 99% in published evaluations [5], machine learning detectors face an operational challenge: **application-domain shift**. An enterprise or cloud WAF must inspect queries across multiple backend database systems exhibiting disparate table schemas, naming conventions, column data types, and query template topologies [6]. When an ML detector trained on one application schema (e.g., an e-commerce catalog) is deployed to protect an unseen application schema (e.g., a corporate human resources system), the distribution of underlying benign queries changes fundamentally, even if attacker payload syntax remains largely invariant [6].

Concurrently, the cybersecurity domain has embraced post-hoc Explainable AI (XAI) frameworks —predominantly SHAP (SHapley Additive exPlanations) [7] and LIME (Local Interpretable Model-agnostic Explanations) [8]—to assist security analysts in triaging alerts and verifying that models do not rely on spurious dataset artifacts [9]. However, existing security XAI literature overwhelmingly evaluates explanations within the same training distribution [5], [10]. A fundamental methodological question remains underexplored:

> *When an SQL injection classifier transfers to an unseen application domain with high detection performance, do its underlying feature attributions remain consistent with those of an in-domain detector, or do models arrive at correct predictions via shifted feature weightings?*

1.1 Predictive Performance and Explanation Consistency as Distinct Dimensions

In security operations, predictive accuracy and explanatory fidelity serve distinct operational objectives. A classifier may achieve high detection accuracy on a novel domain by latching onto coarse lexical heuristics (e.g., character length or quote counts) that happen to separate injected SQL from benign queries in that particular schema. However, if the model's internal attribution shifts away from genuine vulnerability tokens (e.g., boolean comparison tautologies or comment terminators), security analysts inspecting post-hoc explanations risk being misled during incident investigation [9], [11]. Predictive accuracy measures *what* decision was reached, whereas explanation consistency evaluates *why* that decision was made.

1.2 Research Gap and Objectives

Prior studies either evaluate SQLi detection accuracy across application domains without explainability analysis [6], or examine SHAP/LIME explanations qualitatively on single-domain benchmarks without distribution shift [5]. To address this gap, this paper examines the underexplored intersection of application-domain transfer and local explanation consistency in SQL injection detection using the authentic Superviz26-SQL benchmark [6].

Our objective is to determine whether models maintaining high detection performance across application domains generate consistent feature-level explanations, how attribution agreement varies across linear and non-linear architectures, and what methodological constraints govern domain transfer benchmarks. We formalize our investigation around four targeted research questions (RQ1–RQ4) and testable hypotheses detailed in Section 3.

1.3 Summary of Contributions and Scope

The primary contributions of this paper are:

1. **Empirical Transfer Analysis:** We evaluate a single-source supervised transfer protocol from Domain B (Sakila) to Domain D (Oracle HR) on authentic Superviz26-SQL data, establishing query-level evaluation isolation.
2. **Quantitative Explanation Consistency Evaluation:** Across Logistic Regression, Random Forest, and XGBoost, we quantify signed attribution vector agreement (cosine similarity) and feature-rank correlation (Spearman \rho on signed values) using SHAP, contrasting domain-transfer differences with estimator repeat-fit stability and tabular LIME perturbation repeatability across independent seeds.
3. **Descriptive Evidence of Coexisting High Performance and Rationale Differences:** We show that high detection performance coexists with measurable differences in feature attributions (mean cosine similarities of 0.847–0.906, and Spearman \rho as low as 0.447). In 97.5%–99.5% of queries in the 200-query explanation sample where cross-domain and within-domain models yield identical, correct predictions, classifiers arrive at those decisions via shifted feature weightings.
4. **Methodological Template Census & Exposure Analysis:** We document that all 29 vulnerable application templates in Domain D are represented in the 2,000-query evaluation

set, rendering a template-disjoint supervised within-domain reference infeasible under the fixed protocol. We discuss the implications of within-domain template interpolation versus cross-domain template extrapolation.

2. Related Work

2.1 Machine Learning and Feature Engineering for SQLi Detection

Early approaches to SQL injection detection relied on regular expressions and deterministic finite automata (DFA) [12]. To overcome evasion techniques, researchers turned to machine learning. Qi Li et al. [3] formulated an influential manual feature representation consisting of 32 lexical and syntactic tokens extracted directly from raw query strings, capturing character lengths, special character counts, keyword counts, and syntactic flags (e.g., comment tokens and tautologies). Subsequent works applied deep neural architectures, including token graphs [4], CNNs, LSTMs [13], and pre-trained Transformer language models such as CodeBERT [14]. However, manual lexical features remain widely deployed in production WAFs due to their sub-millisecond extraction latency, low computational footprint, and immediate interpretability [3], [15].

2.2 Benchmarks and Application-Domain Transfer

Evaluating SQL injection classifiers requires realistic benchmarks. Historically, research relied on synthetic, aggregated datasets lacking schema provenance, realistic benign query diversity, or documented attack generation mechanisms [6]. Quetel et al. addressed this limitation by introducing Superviz25-SQL [16] and Superviz26-SQL [6], providing multi-domain corpora generated against authentic web applications: OurAirports (Domain A), Sakila (Domain B), AdventureWorks (Domain C), and Oracle Human Resources (Domain D). Superviz26-SQL formalizes domain transfer, showing that unsupervised detectors suffer performance degradation under cross-domain evaluation [6]. However, the Superviz26-SQL benchmark protocol was originally constructed for unsupervised anomaly detection, leaving supervised domain transfer and model explainability unstudied.

2.3 Post-Hoc Local Explainability in Cybersecurity

XAI methods are essential for opening security "black boxes." Lundberg and Lee introduced SHAP [7], grounding local feature attributions in cooperative game theory (Shapley values). Ribeiro et al. proposed LIME [8], approximating non-linear decision boundaries locally using sparse linear surrogates. In cybersecurity, Le et al. [5] applied SHAP and LIME to tree-based SQLi detectors, reporting high detection accuracy and visualizing feature impact. Nugraha et al. [11] combined ANOVA feature weighting with SHAP explanations for network intrusion detection. However, prior security XAI studies examine explanations qualitatively or within the same training distribution [5], [10]. They do not measure attribution divergence when classifiers operate across disparate application domains.

2.4 Explanation Stability and Distribution Shift

A growing body of foundational ML research investigates explanation stability. Studies have demonstrated that post-hoc explainers can be sensitive to input perturbations [17] and distribution shifts [18], [19]. Rajhans and Khawarey [20] introduced the Explainability Stability Index (ESI) to measure TreeSHAP drift under continuous feature perturbations in cybersecurity classifiers. A critical conceptual distinction separates perturbation studies from domain transfer: adversarial perturbation studies [17], [20] alter input queries directly to evaluate explainer sensitivity under malicious or noisy feature transformations. In contrast, application-domain transfer keeps target query instances intact and evaluates whether classifiers trained on distinct database schemas and application environments generate concordant feature rationales for the same target queries. This study examines this latter question using authentic multi-domain benchmark data.

3. Research Questions and Hypotheses

We operationalize our investigation through four formal research questions:

- **RQ1 (Detection Transferability):** *How do detection performance metrics (Accuracy, Precision, Recall, F_1, ROC-AUC, PR-AUC) compare between a cross-domain classifier (M_{\text{cross}} trained on Domain B) and a within-domain reference (M_{\text{within}} trained on Domain D) when evaluated on the identical held-out Domain D evaluation set?*

Hypothesis 1 (H_1): Because manual lexical features capture domain-invariant attack syntax (e.g., `UNION`, comments, tautologies), M_{\text{cross}} will maintain detection performance within 5% F_1 of M_{\text{within}}.

- **RQ2 (Attribution Divergence):** *How similar are signed SHAP feature attributions and feature rankings between M_{\text{cross}} and M_{\text{within}} for identical held-out query instances?*

Hypothesis 2 (H_2): Differing benign token distributions across application schemas will result in measurable differences in feature attributions between M_{\text{cross}} and M_{\text{within}}, yielding attribution cosine similarities below estimator repeat-fit baselines.

- **RQ3 (Architectural Sensitivity):** *How do explanation consistency and feature-ranking stability under domain transfer vary across linear models (Logistic Regression), bagged ensembles (Random Forest), and gradient-boosted trees (XGBoost)?*

Hypothesis 3 (H_3): Ensembles with randomized feature subsampling (Random Forest) will exhibit higher feature-ranking stability than greedy split-finding models (XGBoost) or linear models.

- **RQ4 (Template Exposure Asymmetry):** *What structural constraints and template exposure characteristics govern within-domain versus cross-domain evaluation in Superviz26-SQL, and can a template-disjoint within-domain reference be constructed?*

Hypothesis 4 (H_4): In Domain D, vulnerable attack endpoints are concentrated across a small template vocabulary, creating an exposure asymmetry between within-domain interpolation and cross-domain extrapolation.

4. Dataset and Experimental Methodology

4.1 Benchmark Provenance and the Official Partition Anomaly

We conduct all experiments using the authentic Superviz26-SQL benchmark release Version 1.0 (Zenodo DOI: `10.5281/zenodo.19627322`) [6]. While Zenodo subsequently listed an updated record (`10.5281/zenodo.21068333`), all files, query counts, and schema partitions in this paper correspond strictly to the documented Version 1.0 release. A pre-implementation audit across the raw data files confirmed that **the official benchmark `train` partition contains strictly zero attack queries and exactly 100,000 normal queries per scenario**. All attack queries (~100,600 per scenario) reside exclusively in the official `test` partition.

This structure reflects Superviz26-SQL's original design as an *unsupervised anomaly detection* benchmark [6]. Consequently, fitting supervised binary classifiers directly using the official `train` partition for benign queries and the official `test` partition for attack queries would introduce severe partition-provenance confounding (models would learn partition-specific generation artifacts rather than SQL injection syntax). To eliminate this confounding, our supervised protocol draws both benign and attack queries from the official `test` partition stream (`original_split == "test"`), where benign and attack records were generated under identical environment conditions.

> ***Methodological Disclosure:*** *Drawing both training and evaluation instances from the official test partition is a necessary adaptation to construct a supervised binary classification experiment from an unsupervised benchmark. Although exact-query and normalized-query overlap is verified to be strictly zero, training and evaluation records originate from the same high-level test data generation campaign. We explicitly disclose this sampling limitation and analyze its implications in Section 7.*

4.2 Application Domains

We evaluate single-source transfer between two distinct application domains:

- **Source Domain B (Sakila):** A MySQL database modeling a DVD rental store. Schemas feature entity tables (`film`, `actor`, `rental`, `customer`, `inventory`) with alphanumeric text columns and descriptive attributes.
- **Target Domain D (Oracle Human Resources / OHR):** An enterprise database modeling corporate personnel structures (`employees`, `departments`, `jobs`, `locations`). Queries feature numeric identification codes, date calculations, and administrative endpoints.

4.3 Feature Representation: Active Lexical Tokens (`benchmark_32`)

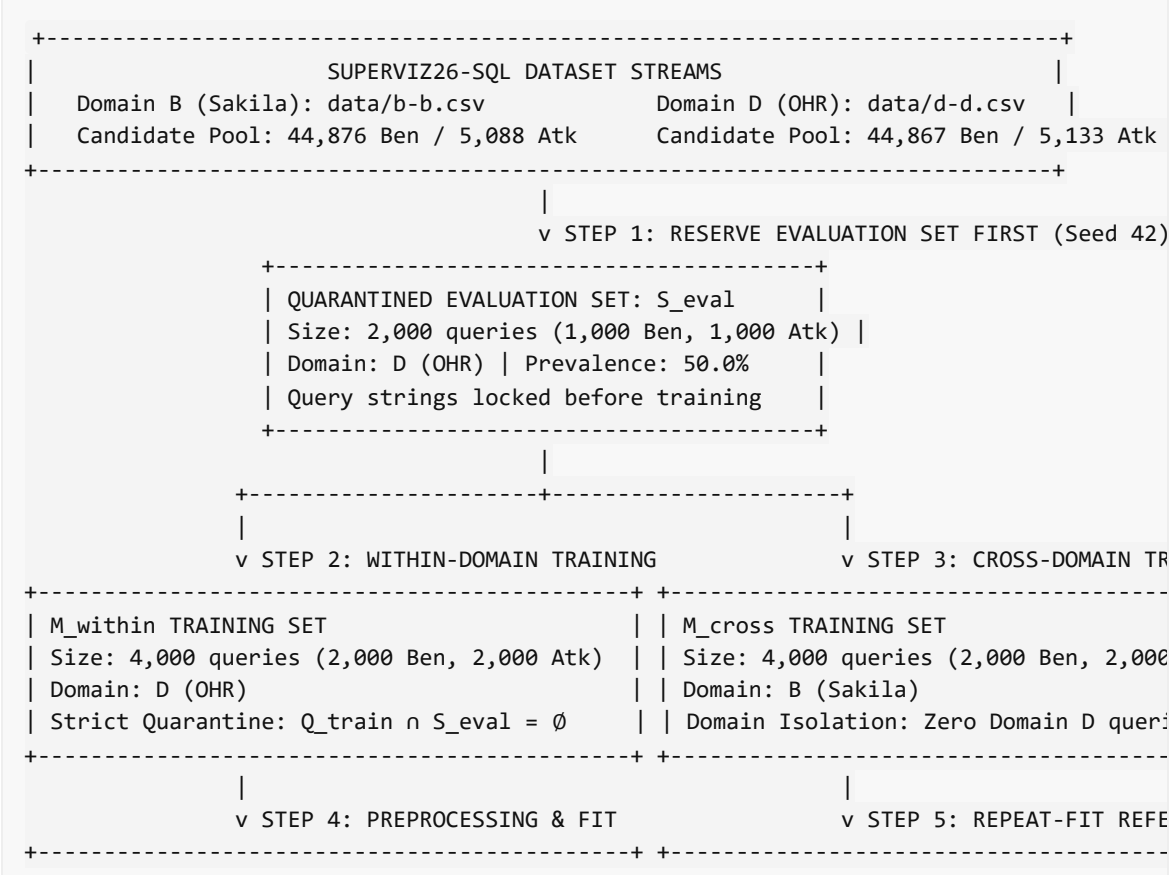
We port the verbatim 32 active manual lexical features formulated by Qi Li et al. [3] and implemented in `U_Li.py` of the official dataset repository. These features capture surface query characteristics without relying on external DBMS parsers:

1. Basic lengths: `len_query`.
2. Character counts: `c_num` (digits), `c_upper` (uppercase letters), `c_space` (whitespace), `c_special` (special characters), `c_arith` (arithmetic operators: `+`, `-`, `*`, `/`, `%`), `c_comparison` (`=`, `<`, `>`, `<=`, `>=`, `<>`, `!=`), `c_square_brackets`, `c_round_brackets`, `c_curly_brackets`.
3. Keyword counts: `has_query_keywords` (`select`, `update`, `insert`, `delete`), `has_database_keywords` (`schema`, `database`, `table`, `column`), `has_connection_keywords` (`and`, `or`, `xor`, `not`), `has_file_keywords` (`load_file`, `into outfile`), `has_exec` (`exec`, `execute`).
4. SQL function tokens: `has_null`, `has_string_functions` (`substring`, `substr`, `mid`), `has_exist_keyword`, `has_floor`, `has_rand`, `has_group`, `has_order`, `has_length`, `has_ascii`, `has_concat`, `has_if`, `has_count`, `has_sleep`.
5. Syntactic attack flags: `has_union`, `has_comment` (`--`, `#`), `has_multiline_comment` (`/`, `/`), and `has_tautology` (regular expression `(\w+)=\1`).

Feature extraction is deterministic. All features are standardized using `StandardScaler` fitted exclusively on training sets.

4.4 Experimental Sampling and Isolation Protocol

To ensure valid comparisons, we implement the experimental protocol shown in Fig. 1:



```
| StandardScaler fitted on X_train only       | | M_cross^(2) REPEAT-FIT MODEL
| Estimators: LR, Random Forest, XGBoost      | | Identical 4,000 Domain B training que
| Random Seed: 42                             | | Random Seed: 101 / 123
+---------------------------------------------+ +-------------------------------------
```

Fig. 1: Experimental protocol architecture enforcing frozen evaluation quarantine and provenance-matched sampling.

1. **Quarantined Evaluation Set (\mathcal{S}_{\text{eval}}):** We sample 2,000 queries (1,000 benign, 1,000 attack; prevalence = 0.500) from Domain D's test partition using random seed `42`. This evaluation set is locked before any training set is constructed.
2. **Within-Domain Training Set (M_{\text{within}}):** We sample 4,000 queries (2,000 benign, 2,000 attack) from the remaining Domain D test pool. Exact query overlap (\mathcal{Q}_{\text{train}} \cap \mathcal{Q}_{\text{eval}}) and normalized query overlap are verified to be strictly zero.
3. **Cross-Domain Training Set (M_{\text{cross}}):** We sample 4,000 queries (2,000 benign, 2,000 attack) from Domain B's test partition using seed `42`. Domain D records are strictly isolated.
4. **Repeat-Fit Reference (M_{\text{cross}}^{(2)}):** To examine estimator variability under identical training data, we train a duplicate instance of M_{\text{cross}} on the identical 4,000 Domain B queries using a different random seed (`101` for LR/RF; `123` for XGBoost).

4.5 Query-Level vs. Template-Level Separation

A critical distinction in structured security benchmarks is the difference between query-level separation and template-level separation:

- **Query-Level Separation:** Two queries differ in literal string representation, numerical parameters, or payload constants. Both our within-domain and cross-domain training sets achieve strictly zero exact and normalized query overlap with the evaluation set.
- **Template-Level Separation:** Two queries originate from distinct parameterized SQL templates. While M_{\text{cross}} faces entirely novel query templates from Domain D, M_{\text{within}} shares query templates with the evaluation set. As audited in Section 4.9, a template-disjoint supervised within-domain reference is infeasible on Domain D.

4.6 Model Architectures

We evaluate three standard architectures:

- **Logistic Regression (LR):** Scikit-learn implementation with L2 regularization (C=1.0), L-BFGS solver, and maximum iterations = 1,000.
- **Random Forest (RF):** Scikit-learn implementation with 100 decision trees, Gini impurity criterion, maximum depth = 12, and bootstrap row sampling (`bootstrap=True`).
- **XGBoost (XGB):** Gradient boosted trees (`xgboost` 3.4.1) with 100 estimators, learning rate = 0.1, maximum depth = 6, and binary logistic objective (`binary:logistic`).

4.7 Evaluation Metrics

On \mathcal{S}_{\text{eval}}, we record Accuracy, Precision, Recall, F_1-score, Area Under the Receiver Operating Characteristic (ROC-AUC), and Average Precision (PR-AUC), using SQL injection = 1 and benign = 0. We define the signed transfer performance difference as:

\Delta F_1 = F_1(M_{\text{cross}}) - F_1(M_{\text{within}})

4.8 Explainability Analysis Protocols

Local feature attributions are computed on a deterministic, stratified subset of evaluation queries (n=200; 100 benign, 100 attack, seed `42`):

- **SHAP Computation & Comparability:** We compute signed attributions \boldsymbol{\phi}_i \in \mathbb{R}^{32} using `LinearExplainer` for Logistic Regression and exact `TreeExplainer` for Random Forest and XGBoost. Attributions are aligned to class 1 (attack) log-odds/margin. All 32 features share identical definitions and column ordering across models. Feature preprocessing (`StandardScaler`) is fitted strictly on each model's training set to prevent data leakage. Because scalers and background reference distributions are fitted/drawn separately from each model's training partition, observed attribution differences reflect both learned estimator weights and training-set reference baselines; this is explicitly treated as a methodological characteristic of comparing distinct domain-trained estimators.
- **Signed Vector Cosine Similarity:** To measure directional attribution agreement between model A and model B on query i:

S_{\text{cos}}(\boldsymbol{\phi}_i^A, \boldsymbol{\phi}_i^B) = \frac{\boldsymbol{\phi}_i^A \cdot \boldsymbol{\phi}_i^B}{\|\boldsymbol{\phi}_i^A\|_2 \|\boldsymbol{\phi}_i^B\|_2}

Edge cases are handled explicitly: if both vectors are all zeros, S_{\text{cos}} = 1.0; if exactly one is all zeros, S_{\text{cos}} = 0.0.

- **Signed Spearman Rank Correlation:** To evaluate whether models prioritize the same relative ordering of features, we compute the Spearman rank correlation \rho(\boldsymbol{\phi}_i^A, \boldsymbol{\phi}_i^B) between signed feature-attribution values. Direction is preserved: positive contributions occupy high ranks, while negative contributions occupy low ranks.
- **Statistical Aggregation and Dispersion Definition:** For each metric distribution, we report mean, median, interquartile range (IQR), and 95% percentile bootstrap confidence intervals computed across B = 1,000 bootstrap resamples with query-level resampling. Where parametric dispersion is reported alongside the mean (e.g., in Table 4), it is formally defined as \text{Mean} \pm \text{SD} (population standard deviation, calculated with degrees of freedom \text{ddof} = 0).
- **LIME Repeat Stability:** Using tabular LIME (`lime` 0.2.0.1), we explain n=50 evaluation queries (25 benign, 25 attack), repeating the explanation 10 independent times per query with documented seeds (`420` to `429`). LIME is used specifically to measure the repeatability of local linear surrogate explanations under instance-centered perturbation

(`sample_around_instance=True`), not as an independent cross-validation of SHAP attributions. Pairwise cosine similarities across the 10 runs (45 pairs) are averaged per query first (S_q), and confidence intervals are computed across the 50 query-level observations to prevent pseudoreplication.

4.9 Template Census and Feasibility Audit

An audit of Domain D revealed that the benchmark contains 46 total query templates, of which only 29 support vulnerable attack endpoints. All 29 attack templates are represented in the 2,000-query evaluation set (100\% template coverage). Consequently, excluding evaluation templates leaves zero attack queries in the candidate pool, making a template-disjoint supervised within-domain reference mathematically infeasible under the locked evaluation set.

5. Results

5.1 RQ1: Detection Performance Under Domain Transfer

Table 1 presents the classification performance of all models on the identical 2,000-query Domain D evaluation set (\text{Prevalence} = 0.500).

Table 1: Classification Performance on Quarantined Domain D Evaluation Set (N = 2,000, Prevalence = 0.500).

Architecture	Training Pool	Accuracy	Precision	Recall	F1-Score	ROC-AUC	PR-AUC	Confusion Matrix (TN / FP / FN / TP)
Logistic Regression	Cross (Domain B)	0.9860	0.9990	0.9730	0.9858	0.9949	0.9963	999 / 1 / 27 / 973
	Within (Domain D)	0.9895	0.9990	0.9800	0.9894	0.9977	0.9983	999 / 1 / 20 / 980
	Repeat (Domain B)	0.9860	0.9990	0.9730	0.9858	0.9949	0.9963	999 / 1 / 27 / 973
Random Forest	Cross (Domain B)	0.9900	0.9980	0.9820	0.9899	0.9952	0.9966	998 / 2 / 18 / 982
	Within (Domain D)	0.9750	0.9666	0.9840	0.9752	0.9979	0.9982	966 / 34 / 16 / 984

Architecture	Training Pool	Accuracy	Precision	Recall	F1-Score	ROC-AUC	PR-AUC	Confusion Matrix (TN / FP / FN / TP)
	Repeat (Domain B)	0.9905	0.9980	0.9830	0.9904	0.9945	0.9963	998 / 2 / 17 / 983
XGBoost	Cross (Domain B)	0.9880	0.9980	0.9780	0.9879	0.9947	0.9964	998 / 2 / 22 / 978
	Within (Domain D)	0.9910	0.9990	0.9830	0.9909	0.9987	0.9988	999 / 1 / 17 / 983
	Repeat (Domain B)	0.9880	0.9980	0.9780	0.9879	0.9947	0.9964	998 / 2 / 22 / 978

The empirical comparisons show:

- **Logistic Regression:** Cross-domain F_1 is 0.9858, compared to within-domain F_1 of 0.9894, representing a slight drop of \Delta F_1 = -0.0036.
- **XGBoost:** Cross-domain F_1 is 0.9879, compared to within-domain F_1 of 0.9909, representing a slight drop of \Delta F_1 = -0.0030.
- **Random Forest:** Cross-domain F_1 is 0.9899, compared to within-domain F_1 of 0.9752, representing a higher cross-domain performance of \Delta F_1 = +0.0147. The within-domain Random Forest produced more false positives on the evaluation sample than the cross-domain model (34 vs. 2 false alarms). While this difference is consistent with a possible sensitivity to domain-specific lexical patterns in Domain D, further feature-level and controlled analyses are needed to establish the underlying cause.
- **Overall:** Two models show slightly lower cross-domain F_1, while Random Forest shows higher cross-domain F_1 in this comparison. These differences are descriptive and have not been tested for statistical significance.

5.2 RQ2: Attribution Similarity Under Domain Transfer

Table 2 details the similarity between signed SHAP attribution vectors of M_{\text{cross}} and M_{\text{within}} evaluated on the identical n=200 query subset.

Table 2: SHAP Attribution Similarity Between Cross-Domain (M_{\text{cross}}) and Within-Domain (M_{\text{within}}) Models (n = 200).

Architecture	Mean Cosine Sim	Median Cosine	IQR Cosine	95% Bootstrap CI	Mean Signed Spearman \rho	Median \rho	IQR \rho	95% Bootstrap CI
Logistic Regression	0.8474	0.8559	0.0913	[0.8273, 0.8660]	0.4467	0.4570	0.1059	[0.4385, 0.4547]
Random Forest	0.8706	0.9034	0.0372	[0.8563, 0.8839]	0.8010	0.8391	0.1743	[0.7826, 0.8183]
XGBoost	0.9058	0.9185	0.0449	[0.8941, 0.9151]	0.4523	0.4505	0.2089	[0.4282, 0.4775]

The results indicate measurable attribution differences across application domains:

- In Logistic Regression, directional cosine similarity averages 0.8474, and signed feature-rank correlation averages \rho = 0.4467.
- In XGBoost, directional attribution similarity averages 0.9058, while signed rank correlation averages \rho = 0.4523.
- In Random Forest, directional attribution similarity averages 0.8706, while signed rank correlation averages \rho = 0.8010.
- High positive cosine similarity (0.847–0.906) indicates substantial shared directional alignment in vector space, while lower rank correlations on signed values (\rho \approx 0.45 for LR and XGBoost) indicate shifts in the ordering of signed attribution weights.

5.3 RQ3: Architectural Differences in Attribution Consistency

Comparing metrics across architectures reveals distinct behaviors:

- **Random Forest Preserves Signed Feature Rankings:** While Logistic Regression and XGBoost yield lower signed rank correlations (\rho \approx 0.45), Random Forest retains a substantially higher signed rank correlation of \rho = 0.8010 (median 0.8391). This indicates an observed architectural pattern where randomized ensemble partitioning maintains feature hierarchies across application domains; isolating whether bagging or tree decorrelation is the primary causal mechanism requires multi-domain ablation.
- **Directional Weighting Differences:** Despite higher rank correlation (\rho = 0.8010), Random Forest's directional attribution similarity remains at 0.8706. In this sample, signed feature ordering appears more consistent than the full attribution vectors. This is a descriptive result and should not be generalized beyond the tested models, dataset, and transfer pair.

5.4 Estimator Repeat-Fit Reference vs. Domain-Transfer Divergence

Table 3 compares domain-transfer attribution similarity against repeat-fit baseline similarity (M_{\text{cross}}^{(1)} vs. M_{\text{cross}}^{(2)}).

Table 3: Repeat-Fit Baseline Similarity vs. Domain-Transfer Similarity (SHAP, n = 200).

Architecture	Domain Transfer Cos Sim [95% CI]	Repeat-Fit Cos Sim [95% CI]	95% CI Comparison
Logistic Regression	0.8474 [0.827, 0.866]	0.9960 [0.996, 0.996]	Separate intervals
Random Forest	0.8706 [0.856, 0.884]	0.9802 [0.979, 0.981]	Separate intervals
XGBoost	0.9058 [0.894, 0.915]	1.0000 [1.000, 1.000]	Separate intervals

Repeat-fit similarity provides an empirical reference for variability under the tested setup:

- **XGBoost Determinism:** Repeat-fit XGBoost yields an exact cosine similarity of 1.0000. Under default full subsampling (`subsample=1.0` , `colsample_bytree=1.0`), greedy split construction on deterministic tabular data produces identical decision trees regardless of random seed (`dump1 == dump2` across all 100 trees). TreeExplainer evaluates identical tree paths, resulting in deterministic explanations.
- **Random Forest Bootstrap Variance:** Random Forest yields a repeat-fit cosine similarity of 0.9802, reflecting tree variance introduced by bootstrap row bagging.
- **Interpretation:** Repeat-fit similarity indicates that model-fitting variance under fixed data is lower than the attribution differences observed between cross-domain and within-domain models. However, this does not isolate application-domain shift from template exposure or other data characteristics.

5.5 LIME Perturbation Stability

Across n=50 evaluation queries with 10 independent perturbation repeats per query (500 total explanations per architecture), mean pairwise cosine similarity is **0.8401** for Logistic Regression (95% CI [0.7802, 0.8962]), **0.9630** for Random Forest (95% CI [0.9585, 0.9671]), and **0.9173** for XGBoost (95% CI [0.8926, 0.9404]). Tabular LIME perturbations on manual lexical features exhibit high local consistency (>0.84) under repeated runs when continuous sampling is centered around the explained instance (`sample_around_instance=True`). Because LIME repeatability and SHAP domain-transfer divergence arise from different procedures, sample sizes, and explanation algorithms, they serve as separate reference points rather than directly comparable quantities.

5.6 RQ4: Stratified Performance Decoupling Analysis

To investigate whether attribution differences are confined to misclassified queries, we partition all n=200 evaluated queries into four mutually exclusive categories (Table 4):

1. **Category 1 (Concordant Correct):** Both M_{\text{cross}} and M_{\text{within}} correctly classify the query instance.
2. **Category 2 (Cross Correct, Within Wrong):** M_{\text{cross}} correctly classifies the query, while M_{\text{within}} misclassifies it.
3. **Category 3 (Cross Wrong, Within Correct):** M_{\text{within}} correctly classifies the query, while M_{\text{cross}} misclassifies it.

4. **Category 4 (Both Wrong):** Both models misclassify the query instance.

Table 4: Mutually Exclusive Prediction Strata Breakdown of SHAP Cosine Similarity (n = 200). Values in cells are reported as Mean \pm SD (population standard deviation, \text{ddof} = 0) alongside sample counts.

Architecture	Category 1: Both Correct	Category 2: Cross Only	Category 3: Within Only	Category 4: Both Wrong	Total Partition Count
Logistic Regression	0.8475 ± 0.1398 (n = 199, 99.5%)	0.8369 (n = 1)* (n = 1, 0.5%)	N/A (n = 0) (n = 0, 0.0%)	N/A (n = 0) (n = 0, 0.0%)	**200 / 200**
Random Forest	0.8784 ± 0.0875 (n = 195, 97.5%)	0.4964 ± 0.0070 (n = 4)* (n = 4, 2.0%)	0.8528 (n = 1)* (n = 1, 0.5%)	N/A (n = 0) (n = 0, 0.0%)	**200 / 200**
XGBoost	0.9132 ± 0.0396 (n = 197, 98.5%)	N/A (n = 0) (n = 0, 0.0%)	0.4212 ± 0.1232 (n = 3)* (n = 3, 1.5%)	N/A (n = 0) (n = 0, 0.0%)	**200 / 200**

\Note: Cell values are reported as Mean \pm SD (population standard deviation, calculated with degrees of freedom \text{ddof} = 0). Strata with very small sample sizes (n = 1, 3, or 4) report individual values or small-sample descriptive averages and cannot support general statistical conclusions.*

In the 200-query explanation subset, 97.5%–99.5% of queries fall into Category 1 (concordant correct). For these queries, attribution cosine similarity remains 0.848–0.913, showing that classifiers achieve identical, correct decisions despite shifted feature rationales. In the few queries where one model errs (Categories 2 and 3), attribution similarities vary: for Random Forest Category 2 (n=4), similarity drops to 0.4964, and for XGBoost Category 3 (n=3), similarity drops to 0.4212. Because these sample sizes are very small, they serve only as descriptive observations. These concordance percentages apply strictly to the 200-query explanation subset and are not generalized to the full 2,000-query evaluation set.

6. Discussion

6.1 Coexistence of High Accuracy with Attribution Differences

Our findings demonstrate an important empirical phenomenon: **detection performance alone did not characterize the observed differences in SHAP attribution consistency in this evaluation**. In Domain D, attack tokens (e.g., `UNION SELECT`, `' OR '1'='1'`, comment dashes) are sufficiently distinctive in lexical space that cross-domain models achieve >98.5\% F_1. However, the models do not weigh those tokens identically to in-domain detectors. In Sakila (Domain B), queries involve text-heavy film descriptions and customer names, influencing how features such as `c_comparison`, `c_space`, and character ratios are weighted. In Oracle HR (Domain D), employee

queries involve numeric identifiers and dates, shifting in-domain feature importance toward `c_special`, `has_tautology`, and keyword frequencies.

6.2 The Random Forest Rank Preservation Phenomenon

A notable architectural observation is the distinction between directional attribution scale and feature hierarchy. For security triage, analysts frequently inspect the top-k most influential features [9]. While Logistic Regression and XGBoost undergo substantial changes in their signed feature rankings under domain transfer (\rho \approx 0.45), Random Forest preserves relative feature hierarchies (\rho = 0.8010) even while shifting numerical weights (S_{\text{cos}} = 0.8706). For security deployments requiring consistent relative feature ranking, bagged ensembles exhibit greater rank correlation than gradient-boosted trees in this setting.

6.3 Methodological Implications of Template Exposure Asymmetry

A central finding of our pre-implementation audit is the structural asymmetry between cross-domain and within-domain models in Superviz26-SQL. Domain D contains 46 total templates, of which only 29 support SQL injection attacks. Because our 2,000-query evaluation set saturated all 29 attack templates (100% coverage), constructing a template-disjoint within-domain reference was formally impossible under this protocol.

Consequently, the within-domain reference model operates via **template interpolation** (it observed all 29 query templates during training, evaluating on unseen parameter constants and payload strings). In contrast, the cross-domain model operates via **template extrapolation** (it evaluated on query structures and schema entities it had never encountered). This asymmetry indicates that the observed attribution differences reflect not only schema transfer, but also the difference between interpolating across familiar templates versus generalizing to novel templates.

7. Threats to Validity

7.1 External Validity

1. **Single Transfer Pair (N=1):** Our experiment evaluates a single transfer scenario (Sakila \to Oracle HR). While Sakila and Oracle HR represent substantially different database domains (media retail vs. enterprise HR), our findings represent a descriptive case study and cannot be generalized to arbitrary enterprise DBMS environments without broader multi-target evaluation.
2. **Synthetic Attack Generation:** Attacks in Superviz26-SQL were generated by automated tools (SQLMap). Real-world human attackers employing multi-stage evasion techniques may produce different linguistic distribution shifts.

7.2 Internal Validity

1. **Official Partition Sampling:** Because Superviz26-SQL's official training partition contains zero attacks, our supervised training and evaluation sets were sampled from the official `test`

partition stream. While exact and normalized query overlap was verified to be strictly zero, training and evaluation instances share the same high-level data generation campaign.

2. **Template Exposure Asymmetry:** The within-domain reference model had exposure to the 29 evaluation templates during training, whereas the cross-domain model did not. Attribution differences cannot be causally attributed to application schema shift alone without accounting for template exposure.
3. **Confounded Sensitivity Analysis:** In our protocol sensitivity analysis, the unmitigated baseline altered both partition provenance (benign from `train`, attack from `test`) and attack prevalence (50% down to 10%). The resulting 5.84% drop in F_1 reflects their joint effect rather than partition confounding alone.

7.3 Construct Validity

1. **Manual Lexical Representation:** Our study evaluates the 32 manual features of Qi Li et al. [3]. While these features are standard in production WAFs, our conclusions do not necessarily transfer to deep sequence models (e.g., CodeBERT) or graph neural networks.
2. **Attribution Similarity as a Proxy:** Cosine similarity and Spearman rank correlation quantify explanation agreement between models. They do not prove that either model's explanation is ground-truth faithful, causally correct, or helpful to a human analyst.
3. **Signed vs. Absolute Ranks:** Our Spearman correlation operates on signed attribution values, preserving directionality. This measures rank correlation of signed contributions rather than feature-importance magnitude alone.

7.4 Conclusion Validity

Our results support a **descriptive observation** of coexisting high accuracy and explanation differences on this evaluation sample. They do not establish statistical independence or causality between detection performance and explanation consistency across a population of domains.

8. Ethical Considerations and Reproducibility

8.1 Dual-Use and Responsible Disclosure

This research evaluates defensive intrusion detection and model explainability. It does not introduce novel attack payloads, exploitation vectors, or evasion techniques. All attack queries evaluated originate from published, publicly available academic benchmarks (DOI: `10.5281/zenodo.19627322`) [6].

8.2 Reproducibility Guarantee

To guarantee strict reproducibility:

- All random operations are seeded (`master_seed = 42`).
- Exact sampling manifests recording all query indices, domain tags, and split streams are serialized in `results/audit/manifest_provenance_matched.json`.

- All serialized pipelines (`.joblib`), signed attribution matrices (`.csv`), and execution logs are archived.
- The entire evaluation pipeline is validated by 87 automated unit and integration tests (`tests/test_*.py`), executing with 100% pass rate in 1.65 seconds.

9. Conclusion and Future Work

In this study, we investigated the relationship between detection transferability and explanation consistency for SQL injection detection models under application-domain shift. Using the authentic Superviz26-SQL benchmark, we evaluated a single-source transfer protocol from Domain B (Sakila) to Domain D (Oracle HR) across Logistic Regression, Random Forest, and XGBoost.

Our findings reveal that **high predictive accuracy does not imply explanatory consistency under domain transfer**. While classifiers transfer with strong detection performance (F_1 \ge 0.985, with signed transfer differences of -0.0036 for LR, +0.0147 for RF, and -0.0030 for XGBoost), their underlying feature attributions exhibit measurable differences (mean cosine similarity 0.847–0.906, signed Spearman \rho ranging from 0.447 to 0.801). Classifiers achieve identical, correct classifications on novel application domains while relying on shifted feature weightings. Furthermore, we audited the benchmark template ecology, showing that within-domain reference models in Superviz26-SQL operate in a template-interpolating regime that cannot be made template-disjoint under the fixed evaluation set.

Future work will expand this investigation across multi-target transfer pairs, evaluate deep language model representations (e.g., CodeBERT), and explore domain-adaptation techniques to enforce explanation consistency across application boundaries.

References

1. OWASP Foundation, "OWASP Top 10: 2021 — A03:2021 Injection," *Open Web Application Security Project*, 2021. [Online]. Available: https://owasp.org/Top10/A03_2021-Injection/
2. W. G. J. Halfond, J. Viegas, and A. Orso, "A Classification of SQL Injection Attacks and Countermeasures," in *Proc. IEEE International Symposium on Secure Software Engineering (ISSSE)*, Arlington, VA, USA, 2006, pp. 13–15.
3. Q. Li, W. Li, J. Wang, and M. Cheng, "A SQL Injection Detection Method Based on Adaptive Deep Forest," *IEEE Access*, vol. 7, pp. 145385–145394, Oct. 2019, doi: 10.1109/ACCESS.2019.2944951.
4. D. Kar, S. Panigrahi, and S. Sundararajan, "SQLiGoT: Detecting SQL injection attacks using graph of tokens and SVM," *Computers & Security*, vol. 60, pp. 206–225, 2016, doi: 10.1016/j.cose.2016.04.005.
5. T.-T.-H. Le, Y. Hwang, C. Choi, R. W. Wardhani, D. S. C. Putranto, and H. Kim, "Enhancing Structured Query Language Injection Detection with Trustworthy Ensemble Learning and

Boosting Models Using Local Explanation Techniques," *Electronics*, vol. 13, no. 22, p. 4350, Nov. 2024, doi: 10.3390/electronics13224350.

6. P.-F. Gimenez, L. Pautet, G. Quétel, and T. Robert, "Superviz26-SQL: Cross-domain SQL Attack Detection Dataset," Zenodo, 2026, doi: 10.5281/zenodo.19627322.
7. S. M. Lundberg and S.-I. Lee, "A Unified Approach to Interpreting Model Predictions," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017, pp. 4765–4774.
8. M. T. Ribeiro, S. Singh, and C. Guestrin, ""Why Should I Trust You?": Explaining the Predictions of Any Classifier," in *Proc. 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*, 2016, pp. 1135–1144, doi: 10.1145/2939672.2939778.
9. W. Samek, G. Montavon, A. Vedaldi, L. K. Hansen, and K.-R. Müller, *Explainable AI: Interpreting, Explaining and Visualizing Deep Learning*, Springer, 2019, doi: 10.1007/978-3-030-28954-6.
10. B. A. Cumi-Guzman, A. D. Espinosa-Chim, M. G. Orozco-del-Castillo, and J. A. Recio-García, "Counterfactual Explanation of a Classification Model for Detecting SQL Injection Attacks," in *Proc. 32nd International Conference on Case-Based Reasoning Workshops (XCBR'24)*, CEUR Workshop Proceedings, vol. 3708, Mérida, Mexico, July 2024, pp. 49–64.
11. B. Nugraha, A. Jnanashree, and T. Bauschert, "A versatile XAI-based framework for efficient and explainable intrusion detection systems," *Annals of Telecommunications*, vol. 80, no. 11, pp. 1095–1120, Nov. 2025, doi: 10.1007/s12243-025-01118-9.
12. G. T. Buehrer, B. W. Weide, and P. A. G. Sivilotti, "Using Parse Tree Validation to Prevent SQL Injection Attacks," in *Proc. 5th International Workshop on Software Engineering and Middleware (SEM)*, 2005, pp. 106–113, doi: 10.1145/1108473.1108496.
13. M. Alghawazi, D. Alghazzawi, and S. Alarifi, "Detection of SQL Injection Attack Using Machine Learning Techniques: A Systematic Literature Review," *Journal of Cybersecurity and Privacy*, vol. 2, no. 4, pp. 816–835, 2022, doi: 10.3390/jcp2040041.
14. Z. Feng et al., "CodeBERT: A Pre-Trained Model for Programming and Natural Languages," in *Findings of the Association for Computational Linguistics: EMNLP 2020*, 2020, pp. 1536–1547, doi: 10.18653/v1/2020.findings-emnlp.139.
15. S. Afrin, M. A. Elsayed, and N. Zincir-Heywood, "SQL-GENIE: SQL Protection using GENerative Modeling for Anomaly Detection against Injection and Evolved Adversarial Attacks," in *Proc. IEEE 49th Annual Computers, Software, and Applications Conference (COMPSAC)*, July 2025, pp. 506–515, doi: 10.1109/compsac65507.2025.00066.
16. G. Quétel, E. Alata, P.-F. Gimenez, T. Robert, and L. Pautet, "Superviz25-SQL: SQL Injection Detection Dataset," in *Proc. ESORICS 2025 Workshops (ANUBIS)*, Toulouse, France, 2025, Zenodo: 17086037.
17. D. Slack, S. Friedler, C. Chen, and S. Singh, "Fooling LIME and SHAP: Adversarial Attacks on Post hoc Explanation Methods," in *Proc. AAAI/ACM Conference on AI, Ethics, and Society (AIES)*, 2020, pp. 180–186, doi: 10.1145/3375627.3375830.
18. C. Molnar, *Interpretable Machine Learning: A Guide for Making Black Box Models Explainable*, 2nd ed., 2022. [Online]. Available: https://christophm.github.io/interpretable-ml-

book/

19. P. Linardatos, N. Papastefanopoulos, and S. Kotsiantis, "Explainable AI: A Review of Machine Learning Interpretability Methods," *Entropy*, vol. 23, no. 1, p. 18, 2020, doi: 10.3390/e23010018.
20. M. Rajhans and V. Khawarey, "Beyond Gradient-Based Attacks: Adversarial Robustness and Explainability Stability in Cybersecurity Classifiers," *arXiv preprint arXiv:2607.01679*, 2026, doi: 10.48550/arXiv.2607.01679.